MACHINE LEARNING PROJECT

RAG Hallucination Detector

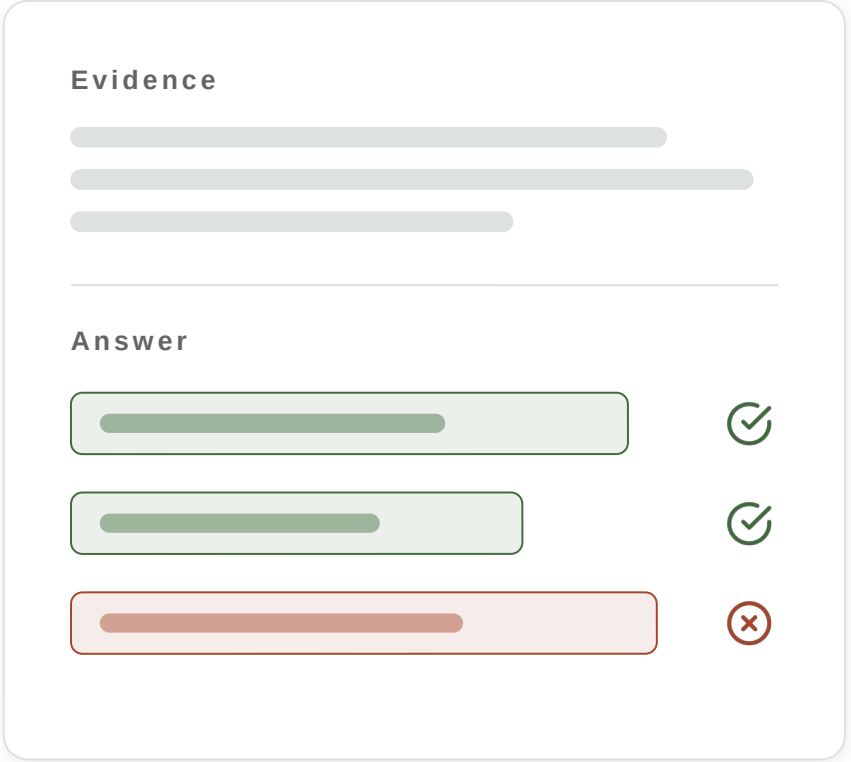
A machine learning approach to identifying unsupported claims in retrieval-augmented generation systems

[Student name]

[University / college] · [Course / subject]

Guide: [Faculty guide] · [Academic year]

github.com/Kavaykhurana/ml_project



Every sentence of an answer, checked against its evidence.

The hallucination problem in retrieval-augmented generation

Retrieval-augmented generation grounds an LLM in retrieved documents. Grounding is not a guarantee: the model can still add entities, numbers or claims the documents never state, or contradict them outright.

WHY IT MATTERS

- Fluent answers earn trust they have not earned
- Retrieved documents do not guarantee factual consistency
- Evaluation usually scores whole answers, not individual claims
- Human verification is slow and does not scale
- One answer can mix correct and hallucinated sentences

"The objective is not merely to generate answers, but to evaluate whether generated answers remain faithful to their evidence."

ILLUSTRATIVE EXAMPLE

Retrieved context

The company was established in 2012 and operates in 15 countries.

LLM response, judged sentence by sentence

The company was established in 2012 and operates in 15 countries.

Supported — both facts appear in the context

It employs 50,000 people worldwide.

Unsupported — the context never states an employee count

GROUNDED

A statement directly supported by the retrieved context.

HALLUCINATED

A statement unsupported by, or in conflict with, the retrieved context.

Example is illustrative. Problem framing: Wu et al., RAGTruth (ACL 2024).

Why RAGTruth? Selecting the right dataset for hallucination detection

17,790
generated responses

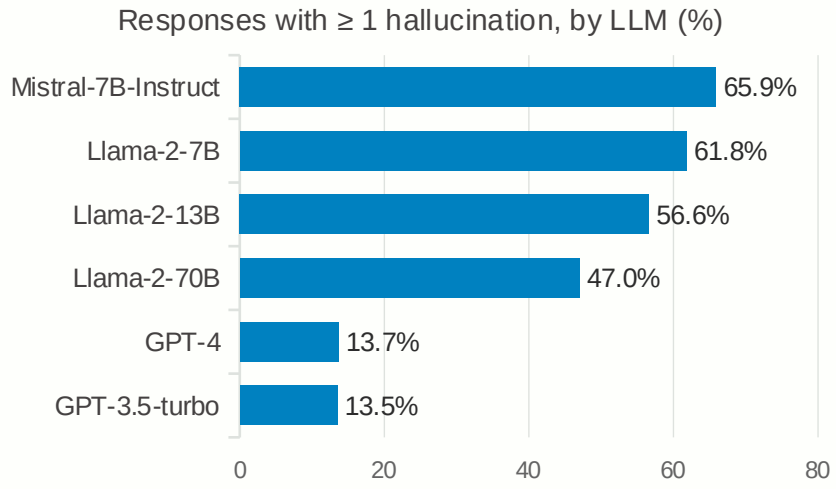
2,965
source instances

14,289
annotated hallucination spans

7,664
responses with ≥ 1 hallucination

6
LLMs from 3 model families

3
task families: QA, data-to-text, summarisation



Criterion	Importance	RAGTruth suitability	Why it matters
Problem relevance	**Critical**	✓ Directly aligned	Built to study hallucinations in RAG outputs
Human-labelled ground truth	**Critical**	✓ Available	Annotators marked hallucinated character spans
Multiple NLP tasks	High	✓ QA · D2T · Summary	Different response structures test generality
Different LLM families	High	✓ Six models	GPT, Llama-2 and Mistral: 13.5–65.9 % hallucinating
Source context	**Critical**	✓ Available	Every response is paired with its retrieved evidence
Reproducibility	High	✓ Official splits	Fixed train / test partition, documented labels

SELECTION RATIONALE

RAGTruth was selected because it provides human-annotated, context-aware examples that directly match the problem of detecting unsupported RAG-generated claims.

Wu et al., RAGTruth (ACL 2024), MIT licence. All counts re-verified on response.jsonl and source_info.jsonl loaded into the project's SQLite database.

Understanding the RAGTruth dataset

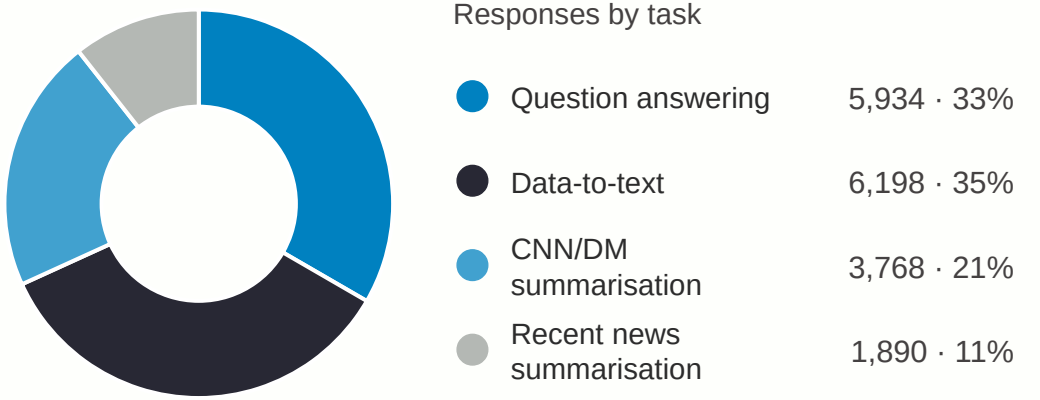
17,790
total responses, full dataset

43.1%
responses with ≥ 1 hallucination

14,289
human-annotated hallucination spans

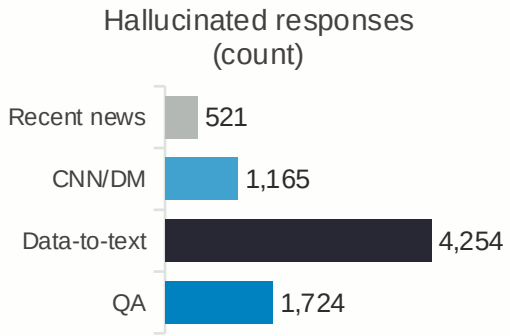
17,617
kept for training after removing 173 refusals / truncations

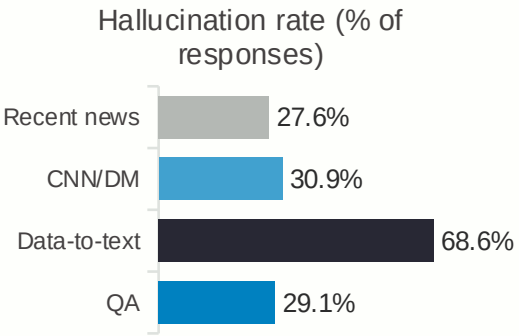
Task	Sources	Responses	Hallucinated	Rate
Question answering	989	5,934	1,724	29.1%
Data-to-text	1,033	6,198	4,254	**68.6%**
CNN/DM summarisation	628	3,768	1,165	30.9%
Recent news summarisation	315	1,890	521	27.6%
Overall	**2,965**	**17,790**	**7,664**	**43.1%**



WHAT THE EDA SHOWS

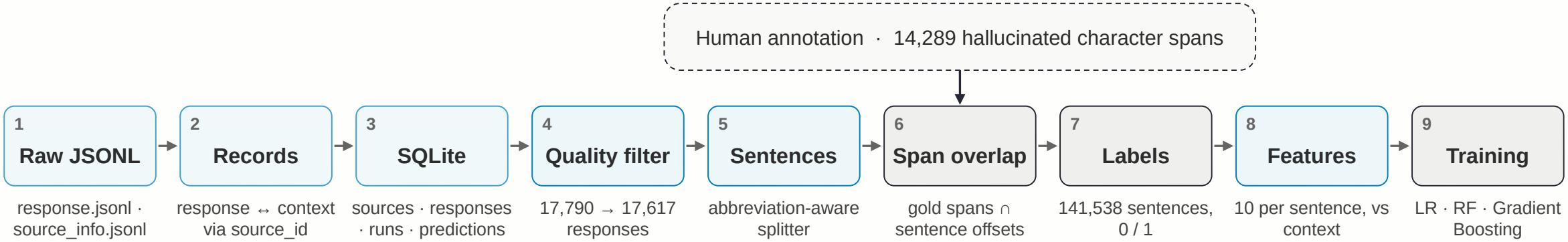
- Data-to-text is the outlier: 68.6% of responses hallucinate, more than twice the QA rate (29.1%).
- Both summarisation sources sit near QA: 30.9% (CNN/DM) and 27.6% (recent news).
- A 2.5× spread across tasks: performance has to be read per task, not from one overall number.





Full RAGTruth release (all splits). The training subset used by the project is smaller — see slide 5. Task-level counts: Wu et al., 2024; totals re-verified on the project database.

From human-annotated responses to machine learning samples



TRAIN / DEV / TEST · CLEANED SUBSET



Split	Responses	Passages	Sentences	Hallucinated sentences
Train	13,531	2,277	110,264	10.8%
Dev (10% of train)	1,411	238	11,092	11.7%
Test (official)	2,675	450	20,182	8.25%

- Dev passages: md5(source_id) % 10 == 0. All six LLM answers to one passage share a split, so no passage leaks between train and dev.
- Model and threshold are chosen on dev; the official test split is used only for the final report.

SENTENCE LABELLING RULE

```
label(s) = 1  if any gold span overlaps s:
span.start < s.end  and  span.end > s.start
```

ILLUSTRATIVE

A · “According to the context, the organization was founded in 2012.”

No overlap with a gold span → label 0, grounded

B · “The organization employs 50,000 people.”

Overlaps a gold span → label 1, hallucinated

Counts from backend.load_dataset() and backend.sentence_table(), verified on the project database. Labelling rule: backend.sentence_table().

Feature engineering — turning text into evidence-based signals

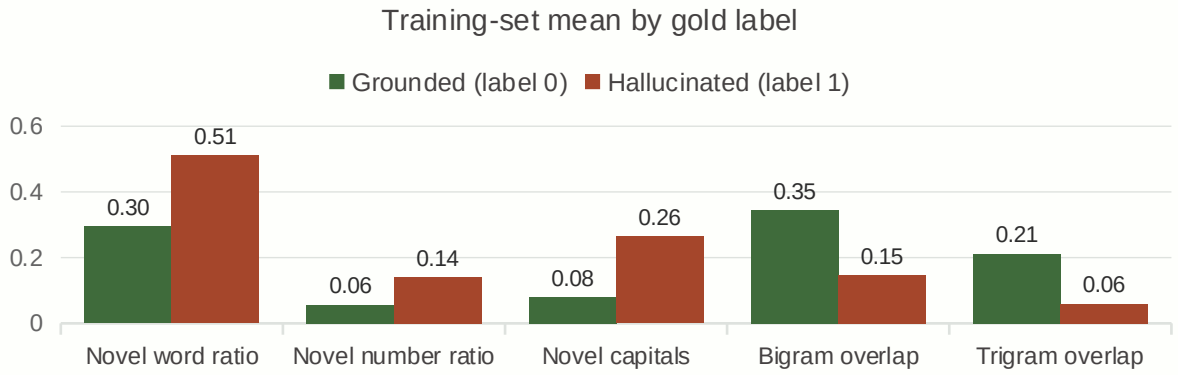
A **Lexical novelty** `novel_word_ratio · novel_words`
Content words in the sentence that never occur in the context

B **Numeric novelty** `novel_numbers · novel_number_ratio`
Numbers and dates that are absent from the context

C **Entity novelty** `novel_capitals`
Capitalised words, likely names, absent from the context

D **Phrase overlap** `bigram_overlap · trigram_overlap`
Share of 2- and 3-word phrases copied from the context

E **Structure** `n_words · position · is_last`
Sentence length and where it sits in the answer



WORKED EXAMPLE · REAL FEATURE VALUES

Context
The company was founded in 2012 and operates in 15 countries.

Sentence 1 · grounded
The company was founded in 2012 and operates in 15 countries.

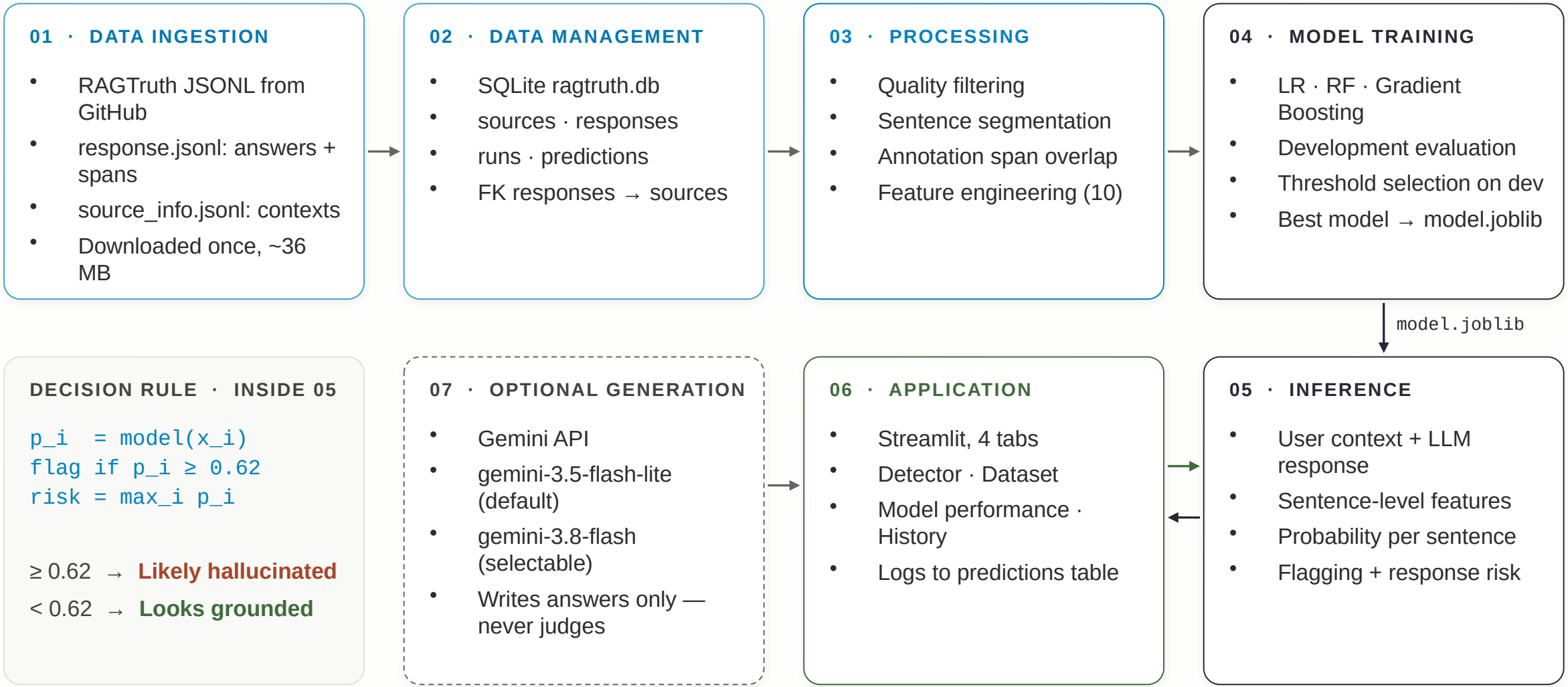
Sentence 2 · unsupported
The company was founded in 2012 and **employs 50,000 workers**.

Feature	Sentence 1	Sentence 2
`novel_word_ratio`	0.00	**0.57**
`novel_words`	0	**4**
`novel_numbers`	0	**2 (50, 000)**
`bigram_overlap`	1.00	**0.60**
`trigram_overlap`	1.00	**0.56**
`novel_capitals`	0	0

The tokenizer splits 50,000 into 50 and 000, so two novel numbers are counted.

Feature code: backend.sentence_features(). Training-set means: notebook §6, 110,264 train sentences. Example values computed by the project code on the illustrative pair.

System architecture — from dataset to hallucination detection



Source: backend.py (data, features, training, inference, Gemini client) and app.py (Streamlit). Decision rule: backend.detect().

Comparative model development and selection

Logistic Regression

LINEAR BASELINE

- Linear reference point
- One weight per engineered feature
- Fast to train, easy to interpret

```
StandardScaler → LogisticRegression(
  max_iter=1000,
  class_weight="balanced")
```

Random Forest

BAGGED TREE ENSEMBLE

- Averages many independent trees
- Learns non-linear feature interactions
- More flexible than a linear boundary

```
RandomForestClassifier(
  n_estimators=300,
  min_samples_leaf=5,
  class_weight="balanced")
```

Gradient Boosting

SELECTED

SEQUENTIAL BOOSTING

- Each tree corrects the previous errors
- Highest dev-split F1 of the three
- Selected for the final detector

```
HistGradientBoostingClassifier(
  max_iter=300,
  learning_rate=0.05,
  class_weight="balanced")
```

SELECTION PROTOCOL

1
Fit each model on train
13,531 responses

→

2
Sweep threshold on dev
0.05 → 0.95, step 0.01

→

3
Keep the best dev F1
Gradient Boosting · 0.392

→

4
Report once on test
threshold fixed at 0.62

Why this is sound

Model choice and threshold are both made on the development split. The test split never influences a decision, so the reported test metrics stay an independent estimate.

Source: backend.CANDIDATES and backend.train(). Dev F1 and threshold of the selected model: Project_Summary.pdf, latest run.

Experimental results — measuring hallucination detection performance

RESPONSE LEVEL · SHOULD THIS ANSWER BE TRUSTED?

Model	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.526	0.830	0.644	0.781
Random Forest	**0.539**	0.806	0.646	0.793
Gradient Boosting (chosen)	0.538	**0.836**	**0.654**	**0.800**

SENTENCE LEVEL · WHICH SENTENCE IS UNSUPPORTED?

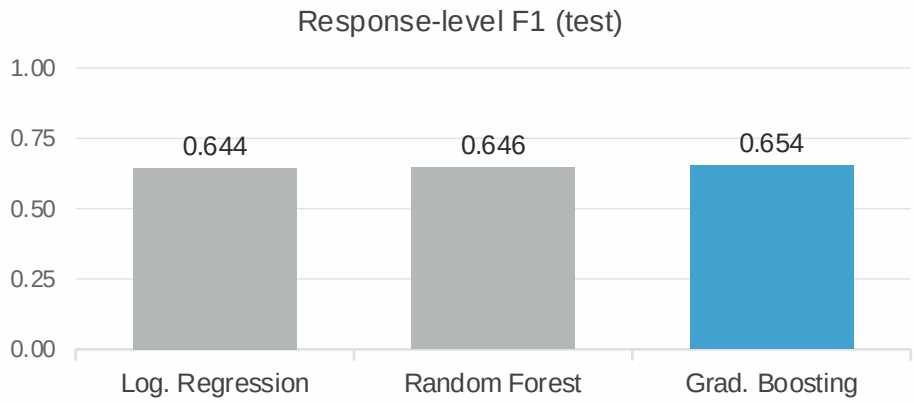
Model	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.211	0.522	0.301	0.771
Random Forest	**0.237**	0.535	0.328	0.784
Gradient Boosting (chosen)	0.235	**0.582**	**0.334**	**0.797**

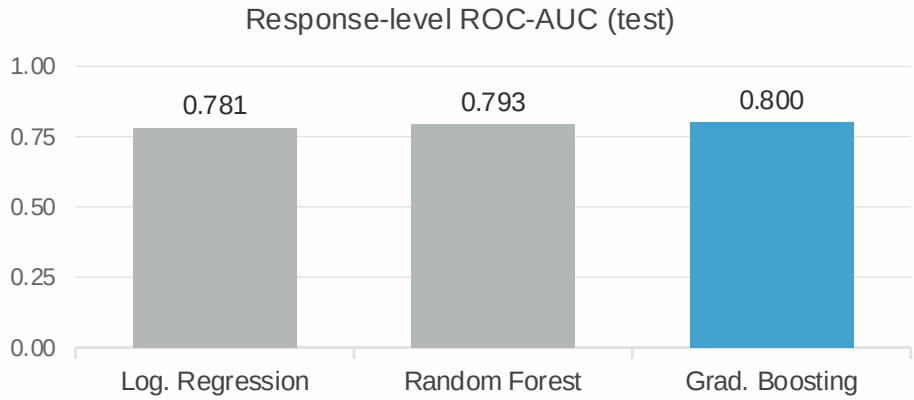
Sentence level

Each sentence is classified. Only 1,665 of 20,182 test sentences (8.25%) are hallucinated, which holds down precision and F1.

Response level

An answer is flagged when at least one sentence is flagged. This is the practical question a user asks.

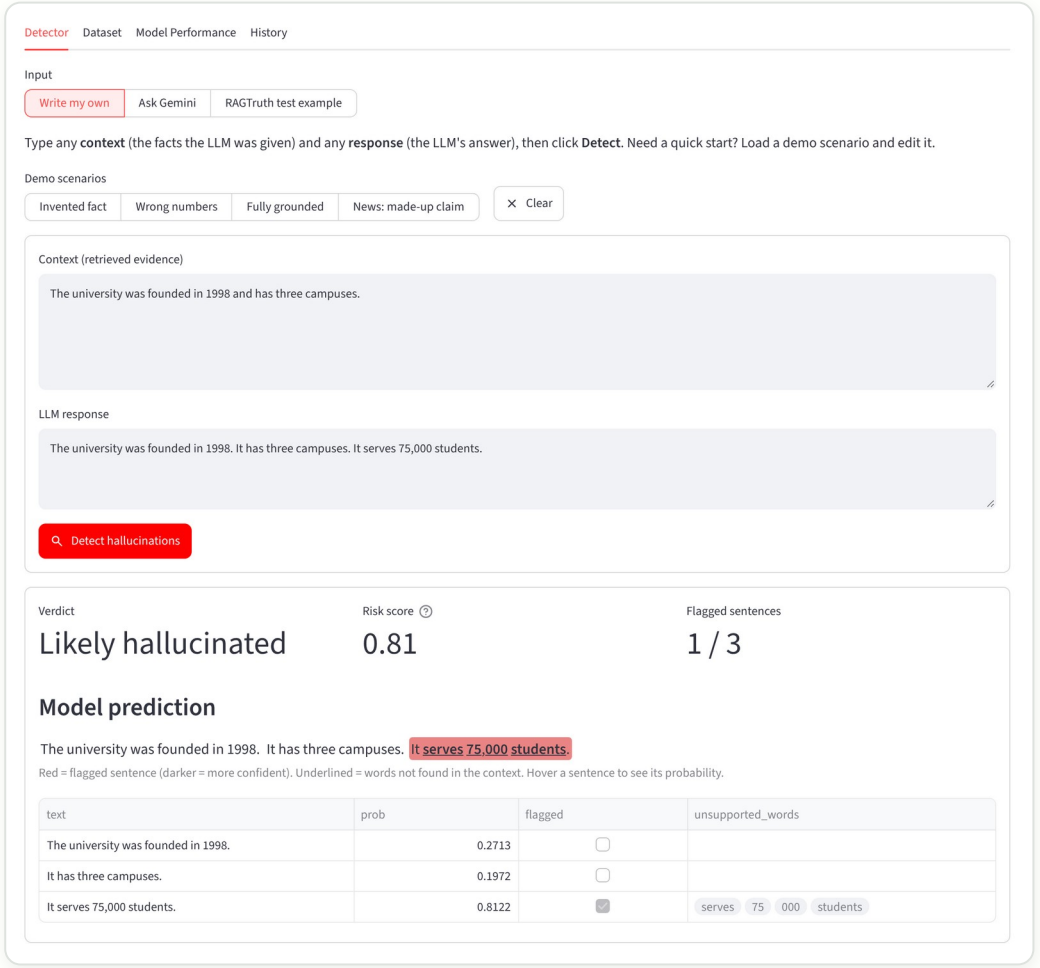




A screening tool, not a sentence oracle: it catches 83.6% of hallucinated answers, while only 23.5% of flagged sentences are truly hallucinated.

Held-out RAGTruth test split: 2,675 responses, 20,182 sentences. Metrics from Project_Summary.pdf (latest run); one run throughout. Best value per column in bold blue.

From machine learning model to working application



Actual screenshot · Detector tab: only sentence 3 is flagged (p = 0.81).

FOUR TABS

Detector

Write my own · Ask Gemini · RAGTruth test example compared with gold labels

Dataset

Live SQLite statistics, EDA charts and a table browser

Model performance

All three classifiers at sentence and response level

History

Every detection: timestamp, risk score, flagged count, verdict

HISTORY TAB · ACTUAL SCREENSHOT



SQLite persistence

sources · responses · runs · predictions

Tested

Backend self-check and 7 end-to-end UI tests

Screenshots: live local run of app.py with a model trained by the project notebook, Nov 2026. Input text is illustrative; probabilities shown are the model's real output.

Critical evaluation — what the model gets right and where it fails

L1 **Lexical, not semantic**

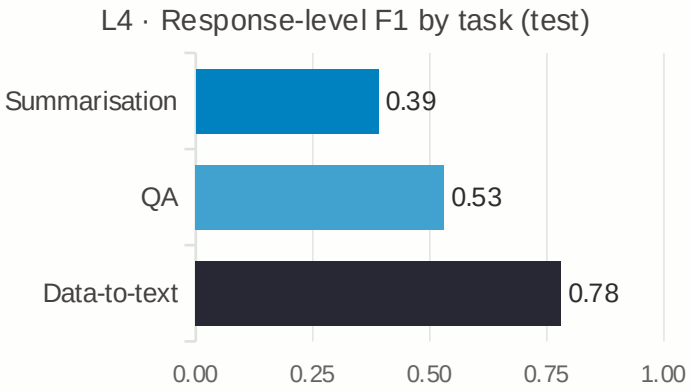
Features count shared words and phrases. The model never reads meaning.

L2 **Paraphrase → false positive**

A correct rewording uses new words, so it looks novel and can be flagged.

L3 **Contradiction → false negative**

Context “Team A defeated Team B.” Response “Team B defeated Team A.” Every word overlaps: the detector returns p = 0.10, looks grounded.



L4 · Task dependence

Data-to-text contexts are JSON, so legitimate rewording looks lexically novel. Its high F1 is driven by flagging almost everything: recall 0.98, precision 0.65.

IMPLEMENTED

Lexical detector

- Word novelty
- n-gram overlap
- Number and entity novelty
- Sentence position

→

PROPOSED

Semantic detector

- Entailment (NLI) score
- Contradiction detection
- Paraphrase-aware matching
- Token-level spans

FUTURE IMPROVEMENTS · NOT YET IMPLEMENTED

1. **Natural language inference**
   add an entailment score as a feature
2. **Transformer detector**
   DeBERTa or ModernBERT on [context] [SEP] [response]
3. **Semantic understanding**
   separate paraphrases from contradictions
4. **Task-specific calibration**
   own threshold for QA, summaries, data-to-text
5. **Generalisation**
   test on other datasets and unseen LLMs

Per-task F1 and Data-to-text precision/recall: project README (test split). Contradiction output: the trained detector run on the shown input.

Conclusion — towards more trustworthy retrieval-augmented generation

KEY CONTRIBUTIONS

- ✓ Context-aware hallucination detection pipeline, end to end
- ✓ Trained on RAGTruth's human-annotated responses
- ✓ Character-span annotations turned into sentence labels
- ✓ Ten lexical and structural features against the context
- ✓ Three classical ML classifiers compared on one protocol
- ✓ Gradient Boosting selected on the development split
- ✓ Streamlit application for practical detection
- ✓ SQLite-backed tracking of runs and prediction history
- ✓ Answer generation (Gemini) kept separate from judgement

NEXT Lexical matching → semantic entailment → transformer-based detection

17,790	3	0.654
dataset responses	ML models compared	response-level F1
0.800	**0.836**	
response-level ROC-AUC	response-level recall	

"An AI response should not be trusted merely because it sounds convincing. Its claims must be evaluated against the evidence that supports them."

This project is an experimental hallucination-screening system. It assists human verification and does not guarantee factual correctness.

REFERENCES

1. Wu et al. RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models. ACL 2024.
2. RAGTruth repository: github.com/ParticleMedia/RAGTruth
3. Project implementation: github.com/Kavaykhurana/ml_project
4. scikit-learn documentation: scikit-learn.org
5. Streamlit documentation: docs.streamlit.io

Metrics: held-out RAGTruth test split, Project_Summary.pdf (latest run).

Thank you

Questions & answers